

단백질 구조 문제의 양자컴퓨팅 접근

고급생명과학 연구보고서

30829 황태준

중앙고등학교

2026-03-29

목차

I. 서론	5
I.1. 연구동기	6
I.1.1. 단백질 구조	6
I.1.2. 양자컴퓨팅	6
I.2. 사전조사 및 선행연구	7
I.2.1. 단백질의 구성 물질과 형성 과정	7
I.2.2. 레빈탈의 역설과 갈때기 모형	8
I.2.3. 시간복잡도와 양자컴퓨팅	12
I.2.4. 단일 양자컴퓨팅	15
II. 본론	17
II.1. 연구내용	18
II.1.1. 수학적 추상화	18
II.1.2. 상태 변수의 정의	20
II.1.3. 해밀토니안 식 세우기	21
II.1.4. 실행 및 출력값 해석	24
II.2. 연구 결과	26
III. 결론	33
III.1. 탐구 후 감상	34
III.1.1. 진행 과정에서 느낀 점	34
III.1.2. 사전 및 사후 조사 과정에서 느낀 점	34
III.1.3. 단백질 구조 연구와 양자컴퓨팅에 대하여	35
III.2. 향후 연구 계획	35

III.2.1. 당장 드는 생각	35
III.2.2. 향후 연구에 대한 사후 조사 결과	36
III.3. 교과 세특 요약	36
참고문헌 및 출처	39

서론 01

I.1. 연구동기

§ I.1.1. 단백질 구조

생명과학 II와 고급생명과학에는 단백질의 형성 과정에 대한 내용이 등장한다. 단백질은 그 입체구조에 의해 기능이 결정되기 때문에 구조가 매우 중요하지만, 20가지의 아미노산 서열*로 이룰 수 있는 3차원 입체구조의 가짓수가 너무 많기 때문에, 특정한 규칙들을 가지고 접히는 그 구조를 예측하는 작업은 품이 매우 많이 든다. 전에는 한 가지 단백질 구조를 알아내기 위해 수많은 생물학자가 달라붙어 많은 시간을 들였으며, 아직도 이렇게 사람이 직접 단백질을 찾고 있다.

하지만 비교적 최근, 단백질 접힘(folding)을 예측하는 AI인 구글 딥마인드†의 알파폴드(AlphaFold)가 등장하였고, AlphaFold2는 꽤 높은 정확도를 자랑하며 약 2억 개에 달하는 3차원 단백질 구조를 예측하는데 성공하였다. 공을 인정받아, 연구에 참여한 공학자 데미스 허사비스(Demis Hassabis)와 생물학자 존 점퍼(John Jumper)가 2024년 노벨 화학상을 수상했다.

이처럼, 단백질은 인간을 포함한 생물의 생리적 작용과 유전 등 모든 방면의 생명활동에서 중요한 역할을 담당하고 있기에, 그 구조를 밝혀내는 것 또한 생물과 화학 분야에서 중요한 연구 과제이다.

§ I.1.2. 양자컴퓨팅

선생님께서 수업시간에 염색체 교차에 대해 말씀하시면서, 이 교차로 인해 유전적 다양성이 증대된다고 하셨다. 교차까지 고려하면 형질이 서로 같은(identical) 두 사람이 태어날 확률은 0에 수렴하며, 어떤 사람이 가질 수 있는 유전자 조합은 슈퍼컴퓨터로도 계산하지 못하는데 다만 양자컴퓨터라면 할 수도 있을지 모르겠다고 하셨다. 여기서 아이디어를 얻어 비록 교차는 아니지만 비슷하게 말도 안되게 큰 경우의 수를 가져 디지털 컴퓨터로 계산해낼 수 없는 단백질 구조에 대한 문제를, 평소에 관심을 가지고 공부하던 양자컴퓨팅으로 해결하려고 시도해 보면 어떨까 생각하게 되었다.

*실제로 생체 내에서 선택 가능한 가짓수는 더 많은 것으로 알려져 있다.

†바둑기사 이세돌 9단과 대결을 펼쳐 4:1로 승리를 거두었던 바둑 AI인 알파고(AlphaGo)를 개발한 사람들이다.

I.2. 사전조사 및 선행연구

§ 1.2.1. 단백질의 구성 물질과 형성 과정

아미노산(amino acid)은 양쪽성 물질로, 중심 탄소 뼈대에 아미노기(amino group—NH₂)와 카르복시기(carboxyl group, —COOH)가 붙어있다. 또, 중심 탄소에 곁사슬(side chain, R)이 붙어있는데, 이 부분이 아미노산의 종류를 결정한다. 아미노산은 곁사슬의 화학적 성질에 따라 크게 네 다섯 가지로 분류할 수 있는데, 아래가 한 가지 예시이다.

- 비극성(非極性), 소수성(疏水性)

글리신(Gly), 알라닌(Ala), 발린(Val), 류신(Leu), 아이소류신(Ile), 메티오닌(Met), 페닐알라닌(Phe), 트립토판(Trp), 프롤린(Pro)

- 극성(極性), 친수성(親水性), 비전하(非電荷)

세린(Ser), 트레오닌(Thr), 시스테인(Cys), 타이로신(Tyr), 아스파라긴(Asn), 글루타민(Gln)

- 산성(酸性), 음전하(陰電荷)

아스파르트산(Asp), 글루타민(Glu)

- 염기성(鹽基性), 양전하(陽電荷)

라이신(Lys), 아르지닌(Arg), 히스티딘(His)

DNA의 이중나선이 풀리고, 유전 정보가 mRNA로 전사되어 핵공을 통해 리보솜으로 이동하여 결합한다. 세 개의 염기 정보가 3염기 조합(triplet code)로서 하나의 코돈을 이루고 이 코돈에 따라 tRNA가 알맞은 아미노산을 가져와 차례로 탈수축합반응(脫水縮合反應)을 통해 펩타이드 결합으로 연결하여 폴리펩타이드 사슬을 형성한다. 종결 코돈을 만나 번역이 끝나고 방출된 폴리펩타이드 사슬을 1차 구조라고 한다. 이 1차 구조 사슬 내부에서 수소 결합 등이 일어나 알파 나선(alpha-helix)나 베타 병풍(beta-sheet) 등의 모양으로 꺾인 것을 2차 구조라고 한다. 2차 구조들이 여러 개 뭉쳐 3차원 구조를 이루는 것을 3차 구조라고 하며, 이때 비로소 단백질이 생리적 기능을 할 수 있게 된다. 3차 구조의 단백

질 덩어리들이 두 개 이상 모여 큰 복합체를 이룬 것을 4차 구조라고 하며, 혈액의 헤모글로빈 등이 그 예이다. [1], [2]

이렇게 결정된 단백질의 구조를 고유 구조(native structure)라고 하고, 이 구조가 단백질의 기능을 결정한다. 즉 단백질이 제대로 된 구조를 이루지 못하면 제 기능을 하지 못한다.

§ 1.2.2. 레빈탈의 역설과 깔때기 모형

단백질 하나가 가질 수 있는 이론적인 구조의 가짓수는 어마어마하다. 예를 들어 100 개의 아미노산으로 된 단백질이 있다고 할 때, 각 아미노산이 가질 수 있는 자유도가 3이라고만 해도 가능한 구조는 3^{100} (약 5×10^{47}) 가지에 달한다. 단백질이 1 초에 10^{12} 번씩 무작위로 모양을 바뀌가며 가장 안정적인 구조를 찾는다고 하면 약 10^{21} 년이 걸려야 한다*. 하지만 실제로 우리 몸 속에서는 단백질이 수 밀리초에서 수 초만에 정확한 구조로 접혀 생성된다. 이것을 『레빈탈의 역설』이라고 한다. 1969년 레빈탈(Cyrus Levinthal)은 이 계산을 통해 단백질은 모든 경우의 수를 다 해보는 무작위의 과정이 아니라, 에너지가 낮은 경로로 유도되며 접힌다는 것을 알아냈다. 이것을 접힘 깔때기 모형(folding funnel model) 모형이라고 한다. [3], [4]

이 『깔때기』의 아이디어를 간략하게 확인해 보자. 흔히 단백질 구조 문제를 논할 때 사용하는 2차원 정방(正方) 격자 단백질 모형(lattice square protein model)을 사용하면 쉽게 논리를 전개해 볼 수 있다. 아미노산은 격자점 위에 놓이는 바둑돌 같은 것이고, 펩타이드 결합은 아미노산들을 잇는 선분이며 한 격자점에 둘 이상의 아미노산이 들어올 수 없다†.

이제 단백질을 구성하는 아미노산의 개수를 N 이라고 하고 계산을 시작해 보자. 어느 격자점에 첫 번째 아미노산이 놓인 후 두 번째 아미노산이 이어질 수 있는 방향은 z 개이다. 2차원 격자에서 $z = 4$ 이 겠지만 실제로는 뒤로 갈 수는 없고 순차적으로 연결되므로 $z = 3$ 이 되어야 한다. 이때 가능한 총 입체 구조(conformation)의 수 Ω 는 점근적으로 다음과 같이 주어진다.

$$\Omega(N) \approx \mu^N N^{\gamma-1} \quad (\text{I.2.1})$$

*우주의 나이가 약 1.3×10^{10} 년이다.

†이것을 통계역학, 고분자 화학에서 SAW(self avoiding walk)라고 하며 파울리 배타 원리 등에 의해 당연하게 받아들여질 수 있다.

이때 μ 는 연결 상수(connective constant) 또는 유효 배위수(effective coordination number)라고 하며, 단백질이 진행하는 도중에 이미 지나온 경로를 뚫고 지나갈 수 없는 것을 반영한다. 예를 들어 10번째 아미노산이 놓일 자리에 이미 2번째 아미노산이 있으면 그 경우는 안 된다는 것이다. 이것을 물리화학자 폴 플로리(Paul Flory)는 배제 체적 효과(excluded volume effect)라고 불렀다. 배제 체적의 개념은 1934년 베르너 쿤(Werner Kuhn)에 의해 처음 제시되었고 곧 폴 플로리에 의해 고분자 이론에 적용되었다. 배제 체적은 액체에서는 반데르발스 상태방정식(Van der Waals equation of state)에서 중요한 변수를 담당하고, 고갈 상호작용/고갈 효과(枯渴相互作用/枯渴效果, depletion force, こかつそうごきよう こかつこうか 枯渴相互作用/枯渴効果)의 원인이 된다*. [5], [6]

이 배제 체적 때문에 실제로 가능한 가짓수는 3개보다 작으며, N 이 커질수록 줄어드는 가짓수가 커짐을 알 수 있다. 이것을 μ 로 나타내는 것이다.

$$\mu = \lim_{N \rightarrow \infty} [\Omega(N)]^{\frac{1}{N}} \quad (\text{I.2.2})$$

수치해석적으로 근사한다면 μ 는 2차원 정방 격자에서 2.638, 3차원 입방 격자에서는 4.684 정도이다.

μ 는 국소적인 스케일에서 진로에 이미 다른 분자가 있는지를 확인한다면 엔트로피 지수(entropic exponent) γ 는 사슬이 아주 길어졌을 때 한참 전에 만들어졌던 사슬에 갇혀서 가지 못하는 경우를 다루는 μ 에 대한 보정치이다. μ 의 값은 격자의 모양 등 미시적 요인에 따라 가변적이지만 γ 는 신기하게도 오직 차원 수에만 의존한다. γ 의 값은 2차원에서는 약 1.344, 3차원에서는 1.157, 4차원 이상에서는 1이다†.

이제 $N = 100$ 일 때를 상정해 본다면, 식 (I.2.1)에 의해 $\omega \approx 10^{41}$ 이고, 단백질의 결합각이 한 번 회전하여 구조가 바뀌는데 걸리는 시간인 1 피코초($\tau \approx 10^{-12}$ s)를 고려할 때 무작위 탐색으로 가장 안정한 구조를 찾는데 걸릴 수 있는 시간 t 는 다음과 같다.

$$t = \tau \cdot \Omega \approx 10^{29} \text{ 초} \approx 10^{21} \text{ 년} \quad (\text{I.2.3})$$

*고갈 효과란 고분자 등의 용액에서 분산된 상대적으로 큰 콜로이드 입자 간에 작용하는 실효적 인력으로, 1954년 아사쿠라 쇼(朝倉昌)와 오사와 후미오(大澤文夫)에 의해 제안되었다. 『고갈』이라는 표현은 일본어 원서에서 그대로 가져온 것이며, 한국어 번역은 찾을 수 없었다.

†4차원부터는 공간이 너무 넓어서 자기 몸통과 부딪힐 확률이 0에 수렴한다.

이것이 레빈탈의 역설의 수학적 아이디어이다. 실제 단백질은 10^{-3} 초에서 1초 만에 제 구조를 찾으므로 단백질이 접힐 때의 에너지 경로는 평평할 수 없다는 것을 유추할 수 있다.

단백질은 서로 다른 20 종의 아미노산 수백 개가 연결된 이종중합체(異種重合體, heteropolymer)이므로, 앞서 본 예시처럼 각 아미노산 간 결합 시 접촉 에너지는 같을 수가 없다. 단백질이 특정 구조로 접혔을 때의 총 에너지 E 는 공간 상에서 접촉해 있는 아미노산 쌍 i, j 간의 상호작용 시 접촉 에너지 ϵ_{ij} 의 합으로 나타낼 수 있을 것이다.

$$E = \sum_{i < j} \epsilon_{ij} \Delta_{ij} \quad (\text{I.2.4})$$

여기서 Δ_{ij} 는 i 와 j 가 맞닿아 있으면 1, 아니면 0을 반환하는 지시함수로 한다. 단백질 간 접촉 에너지가 무작위적이라고 가정한다면 중심극한정리에 의해 정규분포(가우스 분포)를 따르게 될 것이다. 이것을 무작위 에너지 모형(random energy model/REM)이라고 한다.

계가 가질 수 있는 수많은 구조 Ω_0 들의 에너지 E 가 평균이 E_0 이고 분산이 $(\Delta E)^2$ 인 정규분포를 따른다고 하자. 이때 에너지 E 를 가지는 상태 밀도 $n(E)$ 는 총 상태 수에 확률을 곱한 것이다*.

$$n(E) = \frac{\Omega_0}{\sqrt{2\pi(\Delta E)^2}} \exp - \frac{(E - E_0)^2}{2(\Delta E)^2} \quad (\text{I.2.5})$$

이때 엔트로피는 대략

$$S(E) = k_B \ln n(E) \approx k_B \ln \Omega_0 - k_B \frac{(E - E_0)^2}{2(\Delta E)^2} \quad (\text{I.2.6})$$

열역학 제1법칙과 제2 법칙에 의해 $\frac{1}{T} = \frac{\partial S}{\partial E}$ 이므로

$$\frac{1}{T} = -k_B \frac{E - E_0}{(\Delta E)^2} \implies E(T) = E_0 - \frac{(\Delta E)^2}{k_B T} \quad (\text{I.2.7})$$

온도 T 가 낮아지면 에너지는 끝없이 떨어져야 하지만, 미시적 상태가 1개 이하일 수는 없으므로 엔트로피 S 는 0보다 작을 수 없다. 이때 $S(E_g) = 0$ 이 되도록 하는 E_g 에서의 온도 T_g 를 유리 전이 온도

*볼츠만 분포로 도출해도 이 경우 결국 정규분포가 된다.

(glass transition temperature)라고 하고 T_g 로 쓴다^{*†}. [7]

$$T_g = \frac{\Delta E}{\sqrt{2k_B \ln \Omega_0}} \quad (\text{I.2.8})$$

만약 단백질 결합의 에너지 경로가 무작위적으로 「울퉁불퉁」하다면 언젠가 온도가 T_g 이하로 떨어져 국소적 최적해에서 더 이상 내려가지 못하고 갇히게 된다. 강자성체의 스핀 방향이 꼬여 있는 상태를 역학에서는 『절절땀(frustration)』이라고 하는데, 여기에 빗대어 이 상황을 똑같이 절절땀이라고 한다^{‡§}.

그렇다면 어떻게 해야 단백질이 온전하게 그 바닥상태로 고유 구조를 찾아갈 수 있을까? 브링겔슨(Joseph D. Bryngelson)과 윌린스(Peter G. Wolynes)는 고유 구조의 에너지(E_N)는 무작위 에너지 스펙트럼의 하한선(E_g)보다 유의미하게 낮아야한다고 제안했다. 이것이 접힘 깔때기 모형의 수학적 조건이 된다. [4], [8]

$$\delta E = E_g - E_N \gg 0 \quad (\text{I.2.9})$$

깔때기 모형이 깔때기 모형이라고 불리는 이유는, 시각화 시 그래프의 모양이 깔때기를 닮았기 때문이다. 가로축을 엔트로피, 세로축을 에너지로 한다면 수많은 미시적 상태에 있는 아미노산들이 결합을 거듭하여 계의 내부에너지가 낮아지고 결국 그 고유 구조로 모이게 되는 깔때기 모양의 구조를 가진다.

현재 상태가 완벽한 고유 구조에 얼마나 도달했는가를 나타내는 0과 1 사이의 질서 매개변수 Q 를 도입하자. $Q = 0$ 이라면 완전히 풀려 있는 상태이고, $Q = 1$ 이라면 완벽히 접혀 완성된 고유 구조에 도달한 상태이다. 이때 열역학 제2법칙에 의해 계는 자유 에너지가 최소화되는 방향으로 움직인다. 헬름홀츠(Helmholtz) 자유 에너지는 아래와 같이 주어진다.

*고분자 물질이나 유리를 구성하는 규산염(SiO_2) 등을 급속 냉각시키면 분자 배열은 액체 상태 그대로인 채로 분자의 운동에너지가 떨어져 그 자리에 멈춘 상태가 되는 것, 즉 액체 상태에서 점성이 무한대가 되어 고체처럼 딱딱해지는 현상을 유리 전이라고 하고 이때의 온도를 유리 전이 온도라고 한다. 유리를 만들 때 규산염 원료를 달군 다음 급속 냉각(quenching)하는데, 이때 유리전이가 일어나 투명하고 잘 깨지는 고체인 유리가 된다. 규산염이 정상적인 바닥 상태의 고체가 되면 석영 등 결정을 가진 단단한 광물이 된다.

†유리 전이는 「진짜 바닥 상태」가 아니며, 어떤 에너지 강하곡선에서 에너지가 부족해져($T < T_g$) 국소적 최적해(극솟값)에 에너지가 갇혀버린 상태인 것이다.

‡절절땀이란 여러 가지 힘에 의해 바닥 상태로 진화하지 못하고 불안정한 상태로 고정된 것을 의미한다.

§절절땀은 재치 있으면서도 그 현상을 직관적으로 설명하는 정말 좋은 번역어라고 생각된다. 일본어로는 그냥 영어 음을 따와 프러스트레이션(フラストレーション)이라고 하고, 중국어로는 불온정성(不穩定性)이라고 한다.

$$A(Q) = E(Q) - T \cdot S(Q) \quad (I.2.10)$$

Q 가 1에 가까워질수록 올바른 결합들이 형성되며 에너지는 감소해 $E(Q)$ 가 감소하고, 가능한 미시 상태 수가 감소하여 $S(Q)$ 가 감소한다. 즉 두 항이 A 의 값을 줄일지 늘릴지를 놓고 대결을 하는 것이다. 이때 단백질 결합이 중간에 멈추지 않고 $Q = 1$ 까지 혼자서 굴러내려가려면 $Q \in [0, 1]$ 의 모든 구간에서 자유 에너지의 증분 ΔA 가 음수여야한다. 즉, 엔트로피 감소보다 에너지 감소가 훨씬 커야한다는 뜻이다. 이것이 깔때기의 전역적 기울기(global bias)라고 한다.

에너지 감소가 엔트로피 감소를 압도하게 하는 방법을 브링젤슨과 윌린스는 진화라고 설명한다. 생명체들의 유전자가 단백질을 만들다가 찢찢맴 상태에서 멈춰서 요상해지는 일을 줄이고, 궁극적으로 없애기 위해 이 찢찢맴을 최소화하는 서열만을 남겨두었다는 것이다.

이를 바탕으로 특정 아미노산 서열이 제 시간에 접힐 수 있는지 판별하는 척도를 만들 수 있는데, 아래와 같다.

$$\Lambda = \frac{T_f}{T_g} = \frac{\delta E}{\Delta E} \quad (I.2.11)$$

이때 T_f 는 접힘 온도로, 깔때기의 전역적 기울기에 의해 단백질이 고유 구조로 접히려는 성질이 극대화되는 때의 온도를 말하며, 에너지 차이인 δE 에 비례한다. 자연계의 단백질은 보통 $\Lambda \geq 1.6$ 을 만족하며, 그토록 빠르게 최적의 구조를 찾아 완성할 수 있는 생명의 통계역학적 마법이라고 할 수 있다.

앞으로 등장할 컴퓨터와 양자컴퓨팅 부분은 생소한 개념을 설명하는 부분으로, 엄밀함보다는 이 보고서에서 다루는 내용을 이해할 수 있을 정도로만 설명하는 것에 집중했으며 발표할 때에도 비슷하게 설명하고 넘어가도록 하겠습니다. 특히 양자컴퓨팅에 대한 자세한 내용은 2학년 때부터 정리했던 별첨 양자컴퓨팅 탐구보고서에 나와 있습니다. 아무래도 제가 잘 아는 내용이다보니 오히려 글을 자세하고 쉽게 서술하기가 어려운 것 같습니다....

§ 1.2.3. 시간복잡도와 양자컴퓨팅

컴퓨터는 매우 빠른 속도로 정보를 처리하지만, 여전히 「즉시」 처리하는 것이 아니다. 컴퓨터가 문제를 해결하는 데 걸리는 시간은 시간복잡도로 나타낼 수 있고, 빅 O 표기법(big-O notation)을 사용한다.

이해를 위해 예를 들어보자. 1부터 n 까지 자연수의 합을 구하는 문제를 보자. 컴퓨터에게 아래와 같은 명령을 내려 합을 구하게 할 수 있을 것이다.

1. $k \leftarrow 1, S \leftarrow 0$
2. $S \leftarrow S + k$
3. $k \leftarrow k + 1$
4. 만약 $k \leq n$ 이라면 2를 반복, 아니라면 5로 진행
5. s 를 출력

이렇게 한다면 n 이 1 늘어날 때마다 더하는 연산을 수행하는 횟수가 1 늘어나므로 시간복잡도는 $O(n)$ 이라고 쓸 수 있는 것이다. 그런데, 이것보다 더 좋은 방법이 있다. 우리는 아래 공식을 알고 있기 때문이다.

$$\sum_{k=1}^n k = \frac{n(n+1)}{2} \quad (\text{I.2.12})$$

즉 컴퓨터에게 아래 명령을 내리면 똑같은 작업을 n 에 구해받지 않고 같은 시간 안에 빠르게 해낼 수 있다. 이때 시간복잡도는 상수시간으로 $O(1)$ 이다.

1. $\frac{n(n+1)}{2}$ 출력

P(polynomial) 문제란 컴퓨터가 다항 시간 $O(N^k)$ 안에 효율적으로 해답을 구할 수 있는 문제들을 말한다. 정렬, 최단경로탐색, 사칙연산 등이 그 예시이다. 위에서 본 1부터 n 까지 더하는 문제도 P 문제이다.

NP(nondeterministic polynomial) 문제란 다항 시간 안에 주어진 문제에 대한 답이 맞는지 확인할 수 있는 문제를 말한다. 즉, 직접 풀기에는 어려울 수 있지만 답을 받았을 때 확인해 보는 것은 쉬운 문제들을 말한다. 모든 P 문제는 NP 문제에 포함된다. 예시로는 RSA 암호 해독 등이 있다.

NP-Hard 문제란, 적어도 모든 NP 문제만큼은 어렵다는 뜻이다. NP-Hard는 꼭 NP일 필요가 없다. 검산조차 오래걸릴 수 있다는 뜻이다. 외판원 순회 문제(外販員巡廻問題, traveling salesman

problem/TSP)*의 최적해 찾기, 정지 문제(halt problem)† 등이 여기 속한다.

단백질 접힘 문제는 NP-Hard에 속하며, 0 또는 1을 기반으로 결정론적 판별을 수행하는 고전컴퓨터‡의 폰 노이만 아키텍처로는 10^{41} 개의 경우의 수를 확인하려면 시간복잡도가 $O(2^N)$ 이 되므로 우주의 나이보다 긴 시간이 필요, 즉 풀 수 없다.

고전컴퓨터의 비트는 상태 공간이 $x \in \{0, 1\}$ 로 이산적이다. 반면 양자컴퓨터의 큐비트는 힐베르트(Hilbert) 공간 상의 단위 벡터로 정의되며, 양자역학적으로 중첩(重疊, superposition) 상태를 가진다.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad \text{where } |\alpha|^2 + |\beta|^2 = 1 \quad (\text{I.2.13})$$

양자컴퓨터는 고전컴퓨터와 달리 NP 문제에 최적화되어 있다. 이는 큐비트를 여러 개 병렬로 동시에 사용할 때 양자 얽힘(quantum entanglement)이 일어나기 때문이다. N 개의 병렬로 나열된 고전 비트는 특정 순간에 딱 한 가지 상태만을 표현할 수 있다. 예를 들어 01101010 같은 것 말이다. 그러나 N 개의 큐비트가 병렬로 나열되면 파동함수는 2^N 개의 기저(basis) 상태의 중첩으로 표현된다.

$$|\Psi\rangle = \sum_{k=0}^{2^N-1} c_k |k\rangle \quad \text{where } \sum_{k=1}^{2^N-1} |c_k|^2 = 1 \quad (\text{I.2.14})$$

이때 $|k\rangle$ 에서의 k 는 이진수 비트 서열을 십진수로 표현한 것으로, 해당 이진수 상태를 나타내는 상태 벡터를 의미한다. 예를 들어 $|01101010\rangle = |106\rangle$ 이라고 약속한 것이다. 따라서 N 개의 큐비트를 조합으로써 2^N 개의 모든 경우를 동시에 연산 범위로 올려놓고 동시 조작을 하는 것이 가능하다. 이것을 양자 병렬성(quantum parralelism)이라고 한다.

양자컴퓨팅은 연산자(operator) 또는 게이트(gate)라고 불리는 선형변환 행렬을 통해 큐비트의 상태를 조정한 뒤 그 결과를 관측하여 출력을 내놓는다. 대표적인 게이트로는 계를 중첩상태로 두는 아다마르(Hadamard) 게이트, 파울리 X, Y, Z 게이트, 위상 변환 게이트 등등이 있다.

*모든 도시(정점)는 서로 길(간선)로 연결되어 있고 외판원이 모든 도시를 한 번씩 방문해야 할 때, 그 경로를 찾는 그래프 탐색 문제이다.

†임의의 컴퓨터 프로그램과 입력값이 주어졌을 때, 이 프로그램이 실행을 마치고 멈출지 무한히 실행될지를 실행해 보기 전에 미리 판별하는 문제, 또는 그 알고리즘을 구하는 문제이다.

‡트랜지스터로 작동하는 비트 기반의 디지털 컴퓨터를 양자컴퓨터와 비교하여 고전컴퓨터라고 한다.

§ 1.2.4. 단열 양자컴퓨팅

단백질 접힘 문제처럼 최적해를 찾아나가는 최적화 문제(최단강하곡선 문제)는 사실 양자컴퓨터로 풀기에 특화된 문제이다. 양자컴퓨팅에는 그로버(Grover) 알고리즘*과 같이 게이트를 주로 사용하는 게이트/회로 모형 방식 외에도 단열 양자컴퓨팅 방식(adiabatic quantum computing/AQC)이 있다. AQC란 단열정리에 의한 양자컴퓨팅 방식이며 양자 어닐링(quantum annealing)과도 관련이 깊다. [9], [10], [11]

보른(Max Born)과 포크(Владимир А. Фок)가 처음 정립한 내용에 의하면 단열정리는 다음과 같다.

定理 1.1 / 단열정리

가해지는 섭동이 충분히 느리게 작용하고, 고유값과 해밀토니안 스펙트럼의 나머지 부분 사이에 간격이 존재할 경우, 계는 순간의 고유 상태를 유지한다.

직역이라 어렵게 느껴질 수 있지만 우리의 상황에 적용해 쉽게 말하면 어떤 양자계가 바닥 상태에 있을 때 해밀토니안[†]을 시간에 따라 충분히 느리게(단열적으로) 변화시키면 계는 들뜬 상태로 전이하지 않고 계속 각 순간의 바닥상태를 유지한다는 것이다.

이것을 이용한 것이 AQC로, 초기 상태의 해밀토니안을 서서히 최종 상태의 해밀토니안으로 이동시키며 복잡한 최적화 문제를 양자계가 진화할 때 바닥상태로 전이하려는 자연스러운 성향을 이용해 그 해를 구하는 방법이다. 즉, 아래 식에서 $t = T$ 가 될 때, 양자계의 진화에 따라 자연스럽게 바닥상태 최적해인 해밀토니안 H_P 를 관측하게 되는 것이다.

$$H(t) = \left(1 - \frac{t}{T}\right)H_0 + \frac{t}{T}H_P \quad (\text{I.2.15})$$

양자 어닐링이란 이상적인 이론 속의 AQC를 하드웨어로 구현하는 공학적 기법으로, 주변의 열과 소음 등 환경 간섭이 있는 상황에서도 AQC처럼 답을 찾아낼 수 있다. 이때, 양자 터널링(quantum

*정렬되지 않은 데이터베이스에서 특정 항목을 고전컴퓨터보다 획기적으로 빠르게 찾아내는 양자 알고리즘으로, 그로버(Lov Grover)가 1996년에 개발했다.

[†]다들 『Hamiltonian』을 『해밀토니언』으로 쓰는 거 같은데 저는 『해밀토니언』이라고 쓰다가 까먹고 『해밀토니안』이라고 쓰게 돼버려서 그냥 처음부터 해밀토니안으로 쓰겠습니다.

tunneling)이 중요한 역할을 한다. 양자 터널링이란 입자의 파동성(물질파)으로 인해 에너지가 부족함에도 불구하고 에너지 장벽을 뚫고 지나가는 현상을 의미한다. AQC에서는 양자 터널링을 통해 국소적 최적해의 에너지 장벽을 터널링으로 극복하여 전역적 최적해를 더 빠르고 정확하게 찾는 데 도움을 주기 때문에 탐색 속도와 정확도에 큰 영향을 미친다.

본론 02

II.1. 연구내용

§ II.1.1. 수학적 추상화

앞서 상정했던 정방 격자 모형을 먼저 도입하겠다. 정방 격자 모형은 단백질 구조를 통계역학적이거나 수학적으로 연구할 때 널리 쓰이는 방식으로, 단백질의 1, 2, 3, 4 차 구조와 그 구부러짐(이면각)을 단 순화시켜 표현하면서도 실제 단백질 구조 및 형성 과정과 비교했을 때 SAW나 배제 체적 등 중요한 특 성을 반영하므로 크게 일반성을 잃지 않는다. 더불어, 20 가지의 아미노산을 소수성과 친수성으로만 분 류하는 HP 모형(hydrophobic-polar model)을 도입하겠다. HP 모형은 켄 딜(Ken A. Dill)이 1985 년 제안한 모형으로, 이 소수성/친수성 결합 상태가 에너지와 엔트로피에 가장 큰 영향을 주는 요인이 된다. 이렇게 하면, 소수성 단백질끼리 뭉칠 때 에너지가 낮아진다는 규칙을 얻을 수 있다. [12]

단백질이 접히는 과정은 물 속(수용액)에서 일어나는데, 소수성 아미노산들은 접히기 전에는 물에 풀려 있다. 물 분자들은 지속적으로 서로 간 수소 결합을 맺으며 끊임없이 회전하고 이동한다. 즉, 자유 도가 높다*. 하지만 물 분자 옆에 소수성 분자가 있게 되면 물 분자는 소수성 분자와는 결합을 맺을 수 없으므로 소수성 분자를 등지고 물 분자들끼리만 결합을 형성한다. 결과적으로 물 분자들이 소수성 분 자를 둘러싸 가두어 둔 채, 마치 얼음 결정 같은 다면체 모양의 포접구조(包接構造, clathrate cage)를 형성한다†. 이런 것을 소수성 효과(疏水性效果, hydrophobic effect)라고 한다. 이때 포접구조를 형 성한 물 분자들은 자유도가 떨어지므로, 엔트로피 또한 낮아진다. [13], [14], [15]

열역학 제2법칙에 따라 엔트로피는 증가해야하는데, 엔트로피가 증가하는 가장 쉬운 방법 중 하나는 두 소수성 분자가 붙는 것이다. 그렇게 되면 소수성 분자와 물이 맞닿는 표면적이 좁아져 자유도를 잃 는 물 분자들이 적어지게 되고, 따라서 엔트로피가 증가할 수 있다. 그러므로 물 분자는 엔트로피를 「되 찾기 위해」 소수성 분자들을 한 군데로 밀고 가서 서로 붙여버리는데, 이것을 소수성 붕괴(疏水性崩壞,

* 물 분자의 수소결합 때문에 물의 끓는점이 분자량이 비슷한 다른 물질보다 높다거나 얼으면 부피가 커지는 등의 독특한 성질들이 발현한다.

† 흔히 말하는 포접화합물(包接化合物, clathrate compound)는 가스 하이드레이트(메탄 하이드레이트)를 의미하는 것 으로, 바닷속 물 분자들이 메탄을 얼음처럼 둘러싸고 있는 것을 말한다. 미시적으로, 소수성 아미노산에서도 비슷한 일이 일어난다.

hydrophobic collapse) 이론이라고 한다. 정제된 용어로 표현하자면, 극성 용매 내에서 폴리펩타이드나 기타 분자가 취하는 3차원 구조의 형성 과정에서, 소수성 분자와 물이 상호작용하여 가해지는 열역학적 압력으로 분자가 응집되거나 「붕괴」하는 것이다. 이 과정이 반복되면 소수성 아미노산은 단백질 구조의 안쪽으로 몰려 들어가 단단하게 뭉쳐 소수성 핵(hydrophobic core)을 형성하게 되고, 단백질 구조의 바깥쪽은 자연스럽게 친수성 아미노산들이 구성하게 된다. 이런 방식으로 공 모양의 3차 구조가 형성된다.

단백질이 접히는 과정에서 전술한 에너지 및 엔트로피의 변화를 깁스(Gibbs) 자유에너지로 나타내어 알아보자.

$$\Delta G = \Delta H - T(\Delta S_{\text{단백질}} + \Delta S_{\text{물}}) \quad (\text{II.1.1})$$

단백질이 알아서 접히려면 $\Delta G < 0$ 이어야 한다. 단백질이 접혀 나가는 과정에서 내부에 수소 결합과 반데르발스(Van der Waals) 인력이 형성되면서 내부 에너지가 낮아지므로 $\Delta H < 0$ 이고, 단백질들은 특정한 구조를 이루게 되므로 $\Delta S_{\text{단백질}} < 0$ 이다. 그러므로 소수성 아미노산들이 결합하면서 포접 구조가 풀릴 때 증가하는 물의 엔트로피 $\Delta S_{\text{물}}$ 가 더 커야 전체적으로 $\Delta G < 0$ 이 된다.

$\Delta S_{\text{단백질}}$ 을 계산하기 위해 몇 가지 변수를 설정하자. 아미노산의 개수를 N , 풀려 있는 상태에서 자유도를 μ 라고 하고 편의상 3으로 두자. 그러면 풀려 있을 때 가능한 미시상태의 수 Ω_u 는 대략 μ^N 이다. 접힌 후의 구조인 고유 구조의 미시상태 수 Ω_f 는 당연히 단 한 가지이다. 이제 이걸 볼츠만 엔트로피 공식에 대입한다.

$$\begin{aligned} \Delta S_{\text{단백질}} &= S_f - S_u = k_B \ln 1 - k_B \ln \mu^N \\ &= -Nk_B \ln \mu \approx -1.1Nk_B \end{aligned} \quad (\text{II.1.2})$$

즉 아미노산 개당 약 $1.1k_B$ 만큼의 엔트로피가 손실된다.

이제 $\Delta S_{\text{물}}$ 을 구해보자. 전체 아미노산 N 개 중 소수성 아미노산의 비율을 f 라고 하자. 보통 구형 단백질에서 $f \approx 0.5$ 이다. 또, 소수성 아미노산 한 개가 물에 노출될 때 포접구조를 형성하느라 묶이는 물 분자의 개수를 n_w 라고 하자. 실험적으로 n_w 는 대략 10에서 15 사이의 값을 가진다. 이때 단백질이 완전히 풀려 있을 때 묶여있게 되는 전체 물 분자의 개수 W 는 아래와 같다.

$$W = fNn_w \quad (\text{II.1.3})$$

물 분자가 완전히 자유로울 때(bulk water)의 미시상태 수를 Ω_b , 소수성 분자의 영향을 받을 때의 미시상태 수를 Ω_c 라고 하자. 그러면 $\Delta S_{\text{물}}$ 은

$$\Delta S_{\text{물}} = W \cdot k_B (\ln \Omega_b - \ln \Omega_c) = Wk_B \ln \frac{\Omega_b}{\Omega_c} \quad (\text{II.1.4})$$

여기서 물의 나머지 조건(온도, 내부에너지 등)은 일정하므로 Ω 비율은 수소 결합 시 뻗을 수 있는 팔의 방향 등의 가지수로 결정된다. 따라서 이렇게 표현해 보자.

$$\Omega \propto \int_C dt \quad (\text{II.1.5})$$

소수성 분자의 영향을 받을 때는 자유로울 때에 비해 수소 결합 시 팔들을 물과 등지는 방향으로만 뻗을 수 있기 때문에, 실험적 데이터가 없는 상황이므로 그냥 자유로울 때는 C 를 구로, 소수성 분자에 묶일 때는 C 를 반구로 하자. 그러면 Ω 의 비율은 $4\pi r^2$ 과 $2\pi r^2$ 의 비이므로 2이다. 이걸 식 (II.1.4)에 대입하자.

$$\Delta S_{\text{물}} = Wk_B \ln 2 \approx Wk_B \cdot 0.693 = fNn_w k_B \cdot 0.693 \quad (\text{II.1.6})$$

n_w 를 12 정도로 치고 물과 단백질의 엔트로피 증분을 비교하자.

$$\left| \frac{\Delta S_{\text{물}}}{\Delta S_{\text{단백질}}} \right| = \frac{12 \cdot 0.5 \cdot 0.693 \cancel{Nk_B}}{1.1 \cancel{Nk_B}} \approx 3.78 \quad (\text{II.1.7})$$

즉 $\Delta S_{\text{물}}$ 이 $\Delta S_{\text{단백질}}$ 보다 약 4배나 크다. 그러므로 $\Delta G < 0$ 이다. 이 엔트로피 차이가 엔트로피 변화의 가장 큰 원동력이므로, HP 모형만으로 단백질 구조 변화를 간단하게 설명할 수 있다. 그러나, 언제나 모형과 그에 따른 규칙을 구체화하여 알고리즘을 더 실용적으로 만들 수 있음을 명시하는 바이다. 지금은 발상을 제기하는 것에 더 초점을 맞추도록 하겠다.

§ II.1.2. 상태 변수의 정의

양자컴퓨터도 컴퓨터이고, 큐비트가 그 기반이 되므로 입력 변수를 큐비트의 서열로 설정하고 해밀토니안 에너지 상호작용 관계식으로 알고리즘을 표현해야 한다. 따라서 단백질 사슬이 공간 상에서 어떤 구조를 이루고 있는가 하는 위상적 형태를 어떤 큐비트가 0이고 어떤 큐비트가 1인가 하는 이산적 최적화 문제로 바꾸어야 한다.

먼저 아미노산 서열을 입력받아야 할 것이다. 단백질을 구성하는 총 아미노산의 개수를 N 이라고 하고, 각 아미노산의 성질(친수성 P 또는 소수성 H)을 이진화된 s_i 의 순서쌍(또는 N 차원 벡터)로 받는다.

$$S = (s_1, s_2, s_3, \dots, s_N) \in \{H, P\}^N \quad (\text{II.1.8})$$

또, 단백질이 존재할 수 있는 2/3 차원 정방 격자의 총 격자점 개수를 M 이라고 하자. 이제 각 격자점의 좌표를 $\alpha, \beta, \gamma \dots$ 의 인덱스로 나타낸다. 그러면 $\alpha = 1, \dots, M$ 이다. 그리고 공간의 입체성을 나타내기 위해 격자점들을 그래프로 치환해 인접행렬(隣接行列, adjacency matrix) $A_{\alpha\beta}$ 을 사용할 수 있다. 마지막으로 아미노산 i 가 격자점 α 에 위치하는지 여부를 결정하는 이진변수 $q_{i,\alpha}$ 를 도입하겠다. 이것이 큐비트에 대응되는 값이 된다.

이제 $N \times M$ 개의 큐비트가 속하는 상태 공간은 가능한 조합의 가짓수가 $2^{N \times M}$ 개인 거대한 힐베르트 공간이다. $N \times M$ 의 큐비트 행렬을 $\mathbf{Q}$ 라고 하면 $\mathbf{Q}$ 는 아래와 같다.

$$\mathbf{Q} = \begin{bmatrix} q_{1,1} & q_{1,2} & \dots & q_{1,M} \\ q_{2,1} & q_{2,2} & \dots & q_{2,M} \\ \vdots & \vdots & \ddots & \vdots \\ q_{N,1} & q_{N,2} & \dots & q_{N,M} \end{bmatrix} \quad (\text{II.1.9})$$

§ II.1.3. 해밀토니안 식 세우기

이 행렬 $\mathbf{Q}$ 는 아무런 값이나 가질 수 있는 것이 아니다. 생물학적, 물리학적으로 의미가 있는 단백질 사슬로 만들기 위해서는 특정한 규칙들이 있는데, 크게 세 가지를 떠올려 보았다. 이 규칙들을 조작하여 알고리즘이 작동하는 방식을 바꿀 수 있다.

1. 하나의 아미노산은 반드시 공간 상에서 딱 한 곳에 존재해야 한다.
2. 한 개의 격자점에는 단 한 개의 아미노산만이 들어갈 수 있다.
3. 서열 상 이어진 아미노산 i 와 $i + 1$ 은 반드시 인접한 격자 좌표에 있어야 한다.

여기에 추가로 앞서 알아보았던 소수성 효과 규칙을 추가하여 최적화의 방향을 지정해줄 수 있다.

4. 서열 상에서 인접하지 않지만 공간 상으로 인접한 소수성 아미노산끼리 결합하면 에너지가 낮아진다*.

*서열 상에서 인접해 있는 소수성 아미노산들은 어차피 둘이 붙어있어야만 하는 관계이기 때문에 계산의 대상이 되면 비효율적이다.

이 규칙들을 반영하여 해밀토니안 식을 세워보면 아래와 같을 것이다.

$$H_P = \lambda(H_1 + H_2 + H_3) + H_4 \quad (\text{II.1.10})$$

여기서 해밀토니안은 일종의 비용함수(cost function)처럼 작동하고, λ 는 페널티를 주는 상수로 아주 큰 값을 가져서, 해당 규칙을 어기고 얻은 물리적 에너지 이득보다 해당 규칙을 어겨서 받는 페널티가 무조건 커야 한다.

먼저 H_1 부터 세우자. 특정 아미노산 i 에 대해 모든 좌표 α 를 조사했을 때, $q_{i,\alpha}$ 의 값은 항상 1이어야 한다. 입력에서 지정한 아미노산이 없어서도 안 되고, 두 개 이상이어서도 안 된다. 합이 1에서 벗어나면 양의 값으로 크게 보내버리기 위해 제곱을 취하자.

$$H_1 = \sum_{i=1}^N \left[1 - \sum_{\alpha=1}^M q_{i,\alpha} \right]^2 \quad (\text{II.1.11})$$

H_2 를 판별하는 기준은 한 좌표에 아미노산이 동시에 있는지이므로 두 아미노산 i, j 에 대해 $q_{i,\alpha}q_{j,\alpha}$ 의 값이 0인지 1인지를 확인하면 된다. 1이라면 한 자리에 두 개 이상 있다는 것이므로 H_2 의 값에 반영한다.

$$H_2 = \sum_{\alpha=1}^M \sum_{i < j}^N q_{i,\alpha} q_{j,\alpha} \quad (\text{II.1.12})$$

H_3 를 판별하기 위해서는 인접행렬 $A_{\alpha\beta}$ 를 가져오자. 아미노산 순서쌍 $(i, i+1)$ 에 대해 그 어떤 두 아미노산이라도 인접행렬 상에서 1의 값을 반환하지 않는다면 규칙을 어기는 것이 된다.

$$H_3 = \sum_{i=1}^{N-1} \sum_{\alpha=1}^M \sum_{\beta=1}^M (1 - A_{\alpha\beta}) q_{i,\alpha} q_{i+1,\beta} \quad (\text{II.1.13})$$

마지막으로 H_4 에 대해, 서열 상 바로 옆에 붙어있지 않지만 공간 상 인접하는 아미노산을 에너지를 낮춰야 한다. 아미노산 i, j 가 모두 소수성일 때 1, 아니면 0을 가지는 행렬 δ_{ij} 를 도입하자. 이때 $\delta_{ij} = 1$ 이면 값이 작아져야 하므로 $-$ 를 붙여주자.

$$H_4 = - \sum_{i < j-1}^N \delta_{i,j} \sum_{\alpha=1}^M \sum_{\beta=1}^M A_{\alpha\beta} q_{i,\alpha} q_{j,\beta} \quad (\text{II.1.14})$$

해밀토니안 식은 완성되었지만, 이것을 실제로 AQC 기반의 양자컴퓨터가 실행하게 하려면 QUBO 매핑을 수행해야 한다. QUBO(이차 비제약 이진 최적화, quadratic unconstrained binary optimization)이란, 변수끼리의 곱이 최대 2 개까지만 허용되고, 어떤 값에 대한 명시적 제한을 두지 않고 대신 페널티로 제약 조건을 설정하며, 변수는 오직 0과 1의 값만 가져야 하는 상태에서 진행되는 최적화 문제이다. 즉 다음과 같다.

$$\text{Minimize: } y = \sum_i a_i x_i + \sum_{i < j} b_{ij} x_i x_j \quad (\text{II.1.15})$$

$0^2 = 0$ 이고 $1^2 = 1$ 로, 이진변수는 제곱해도 그 값이 원래와 다르지 않다는 점($q^2 = q$)을 이용해 해밀토니안 식을 QUBO의 형태로 정리할 수 있다.

$\sum q =: S$ 로 놓고 식 (II.1.11)를 전개하자.

$$(1 - S)^2 = 1 - 2S + S^2 = 1 - 2 \sum_{\alpha} q_{i,\alpha} + \left[\sum_{\alpha} q_{i,\alpha} \right]^2 \quad (\text{II.1.16})$$

여기서 제곱항은 모든 조합의 곱이다.

$$\left[\sum_{\alpha=1}^M q_{i,\alpha} \right]^2 = \sum_{\alpha=1}^M q_{i,\alpha}^2 + 2 \sum_{\alpha < \beta} q_{i,\alpha} q_{i,\beta} \quad (\text{II.1.17})$$

$q^2 = q$ 이므로

$$H_1 = \sum_i \left(1 - 2 \sum_{\alpha} q_{i,\alpha} + \sum_{\alpha} q_{i,\alpha} + 2 \sum_{\alpha < \beta} q_{i,\alpha} q_{i,\beta} \right) \quad (\text{II.1.18})$$

식을 정리하면

$$H_1 = \sum_i \left(1 - \sum_{\alpha} q_{i,\alpha} + 2 \sum_{\alpha < \beta} q_{i,\alpha} q_{i,\beta} \right) \quad (\text{II.1.19})$$

H_2, H_3, H_4 은 이미 QUBO를 만족한다. 위의 식을 아래 모양과 같이 행렬을 포함한 식으로 간단히 정리할 수 있다.

$$E(\mathbf{x}) = \mathbf{x}^T \mathbf{W} \mathbf{x} \quad (\text{II.1.20})$$

그렇게 하기 위해 먼저 $q_{i,\alpha}$ 형태였던 $\mathbf{Q}$ 행렬을 1차원 벡터로 평탄화할 필요가 있다. 그 벡터를 $\mathbf{x}$ 라고 하는 것이다. $\mathbf{Q}$ 를 $\mathbf{x}$ 로 $k = (i - 1)M\alpha$ 로 매핑하면 $\mathbf{x}$ 는 총 변수의 개수는 $K = N \times M$ 개 이고 행이 K 개인 열벡터가 된다.

$$\mathbf{x} = \begin{bmatrix} q_{1,1} \\ q_{1,2} \\ \vdots \\ q_{N,M} \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_K \end{bmatrix} \quad (\text{II.1.21})$$

이렇게 하면 식 (II.1.20) 는 아래 꼴과 같이 나오게 된다. 이때 $\mathbf{W}$ 는 계산의 편의를 위해 하삼각 성분을 모두 0으로 하는 상삼각 행렬(上三角行列, upper triangular matrix)로 한다.

$$E = \begin{bmatrix} x_1 & x_2 & \cdots & x_K \end{bmatrix} \begin{bmatrix} W_{1,1} & W_{1,2} & W_{1,3} & \cdots & W_{1,K} \\ 0 & W_{2,2} & W_{2,3} & \cdots & W_{2,K} \\ 0 & 0 & W_{3,3} & \cdots & W_{3,K} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & W_{K,K} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_K \end{bmatrix} \quad (\text{II.1.22})$$

이 행렬곱을 전개해보면, 대각 성분은 같은 변수의 곱인 1차항이, 비대각 성분은 서로 다른 두 변수의 곱인 2차항이 되면서 식 (II.1.15) 꼴이 되는 것을 알 수 있다. 행렬 $\mathbf{W}$ 의 성분들에는 앞서 유도했던 H 식들의 계수가 들어간다. 따라서 전개된 꼴은 아래와 같을 것이다.

$$E = W_{1,1}x_1 + W_{2,2}x_2 + \dots + W_{1,2}x_1x_2 + W_{1,3}x_1x_3 + \dots \quad (\text{II.1.23})$$

§ II.1.4. 실행 및 출력값 해석

이 보고서에서 D-Wave 社の 컴퓨터 등과 같은 공학적인 양자컴퓨터 설계 방법 등에 대해 다루지는 않을 것이다. 하지만 본 양자 알고리즘의 최종 목표는 앞서 수학적으로 정의한 $\mathbf{W}$ 행렬을 양자컴퓨터에 주입하고* 양자동역학적 과정을 통해 답을 출력해낸 뒤 이것을 다시 생물학적 의미를 가지는 구조로 복원하는 것이다.

가장 먼저, 모든 K 개의 큐비트에 대해 강력한 횡자장(橫磁場, transverse field)을 걸어 가능한 모든 상태의 중첩 상태로 초기화시킨다.

$$H_0 = - \sum \sigma_x^{(i)} \quad (\text{II.1.24})$$

*아미노산의 서열들을 고전컴퓨터에 입력하면, 고전컴퓨터에서 먼저 서열 정보를 행렬 $\mathbf{W}$ 로 처리하여 양자컴퓨터로 전달한다.

여기서 σ_x 는 파울리 X 연산자이다. 단일 큐비트 파울리 X 연산자 σ_x 는 아래와 같다.

$$\sigma_x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (\text{II.1.25})$$

이 연산자의 고유값 방정식 $\sigma_x|\psi\rangle = \lambda|\psi\rangle$ 의 해를 구해보자. 먼저 특성방정식 $\det(\sigma_x - \lambda I) = 0$ 을 풀자.

$$\begin{vmatrix} -\lambda & 1 \\ 1 & -\lambda \end{vmatrix} = \lambda^2 - 1 = 0 \quad \therefore \lambda = \pm 1 \quad (\text{II.1.26})$$

이제 각 고유값을 $(\sigma_x - \lambda I)|\psi\rangle = 0$ 에 대입하여 $|\psi\rangle$ 를 찾자. $\lambda = 1$ 일 때,

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \therefore x = y \quad (\text{II.1.27})$$

$\lambda = -1$ 일 때,

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \therefore x = -y \quad (\text{II.1.28})$$

정규화조건 $x^2 + y^2 = 1$ 을 만족시키는 $|\psi\rangle$ 를 고려하면 최종적으로 해는 다음과 같다.

$$\lambda = \pm 1, \quad |\psi\rangle = \frac{1}{\sqrt{2}}(|0\rangle \pm |1\rangle) = |\pm\rangle \quad (\text{복부호동순}) \quad (\text{II.1.29})$$

이때 H_0 이 최소이려면 $-\sigma_x^{(i)}$ 의 기댓값이 최소인 -1 이어야 하므로 $\sigma_x^{(i)}$ 의 기댓값은 최대인 1 이 되면서 각 큐비트는 고유값이 1 인 상태, 즉 $|+\rangle$ 상태가 된다. 계의 초기 파동함수 $\psi(0)$ 은 다음과 같이 모든 기저 상태가 동일한 확률 진폭을 갖는 상태로 된다.

$$|\psi(0)\rangle = \frac{1}{\sqrt{2^K}} \sum_{\mathbf{x} \in \{0,1\}^K} |\mathbf{x}\rangle \quad (\text{II.1.30})$$

초기화가 완료되었다면 시간에 따라 해밀토니안을 초기 상태 H_0 에서 아까 세워놓았던 단백질 규칙의 상태 H_P 로 서서히 변화시킨다. 이것을 어닐링 경로(annealing path)라고 한다.

$$H(t) = \left(1 - \frac{t}{T}\right)H_0 + \frac{t}{T}H_P \quad (\text{II.1.31})$$

이때 단일 조건을 만족하기 위해서는, 즉 계가 바닥 상태를 유지하며 가기 위해서는 변화 속도가 매우 느려야 한다. 특히 $1 - \frac{t}{T}$ 와 $\frac{t}{T}$ 가 교차하는 지점인 $t = \frac{T}{2}$ 인 지점에서 에너지 간격이 가장 좁아지므로 이때 계가 들뜬 상태로 튀어나가지 않도록 정교한 속도 제어가 필요하다*.

계가 진화를 마치고 $H = H_P$ 인 상태가 되었다면 $|\psi(T)\rangle \approx |\mathbf{x}_{\text{opt}}\rangle$ 로 최적해의 상태에 도달해 있게 된다. 이제 이것을 측정한다. z 축 기저로 양자상태를 측정하면 중첩되어 있던 파동함수가 붕괴하면서 하나의 상태 벡터가 나온다. 예를 들자면 다음과 같이 생겼을 것이다.

$$\mathbf{x}_{\text{출력}} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} \quad (\text{II.1.32})$$

이제 이 결과를 다시 $N \times M$ 의 행렬 $\mathbf{X}$ 로 변환한다.

$$x_{i,\alpha} = (\mathbf{x}_{\text{출력}})_{(i-1)M+\alpha} \quad (\text{II.1.33})$$

이제 몇 번 아미노산이 몇 번째 격자점에 있는지를 알게 되고, 이것을 다시 2차원 또는 3차원 직교좌표계 값으로 변환하여 표현하면 입력한 아미노산 서열로 만들어지는 단백질의 고유 구조를 알 수 있게 되는 것이다.

II.2. 연구 결과

양자컴퓨팅을 이용해, 아미노산의 HP 서열을 입력하면 그 고유구조를 내놓는 알고리즘을 설계했다. 앞에서는 이 문제를 고전컴퓨터로는 풀 수 없다고 하였으나, 어디까지나 시간복잡도와 공간복잡도가 커지기 때문에 그런 것이지 기하급수적인 자원을 때려박고 우주의 나이의 몇 배가 되는 시간을 기다리면 풀어낼 수 있다. 하지만 그것은 서열의 크기가 큰 수로 늘어났을 때를 말하는 것이고, 고전컴퓨터로

*사실 항상 $1 - \frac{t}{T}$ 와 $\frac{t}{T}$ 이지 않으며, $H(t) = A(t)H_0 + B(t)H_P$ 에서 $A(t)$ 는 1에서 0으로 감소하고 $B(t)$ 는 0에서 1로 증가한다.

짧은 서열을 계산해 내는 것은 가능하다*. 따라서 입력과 결과가 어떨지, 그리고 내부적으로는 어떻게 작동할지를 C++를 이용하여 양자 어닐링을 흉내내어 구현해 보았다.

본 문서 하단이나, Github(<https://github.com/tjwhang/ProteinAQC/blob/main/main.cpp>)에서도 코드를 열람할 수 있다.

싱글빙글 만들고 실행했는데 처음부터 꺾이었다.

```

PS C:\Users\... 026CAP\생2_ProteinQC> cd "c:\Users\...
aqc.cpp -o aqc } ; if ($?) { .\aqc }
도출된 고유구조
아미노산 1(H) ~ (2,3)
아미노산 2(P) ~ (1,3)
아미노산 3(H) ~ (1,4)
아미노산 4(H) ~ (2,4)
아미노산 5(P) ~ (2,5)
아미노산 6(H) ~ (2,4)
.....
...PH.
...HHP
.....
.....
0.00999538
PS C:\Users\... 026CAP\생2_ProteinQC>
  
```

그림 II.2.1: 아미노산 꺾침

아미노산 4와 6이 격자점 (2,4)에서 꺾쳐버린 것이었다. 처음에는 코딩을 잘못된 줄 알았으나, 코드 설계의 문제가 아니라 고전컴퓨터의 한계였다. 고전컴퓨터로 시뮬레이션하는 어닐링은 양자터널링이 없기 때문에 작은 N 값에서도 오류가 많기 때문이다. 그래서 λ 페널티 상수, 시도 횟수 등 수치를 조절하고 결과 출력 후 해당 결과가 타당한지를 확인하는 코드를 추가했다.

```

aqc.cpp -o aqc } ; if ($?) { .\aqc }
=====
[✓] 검증: 물리적으로 유효함
=====
도출된 고유구조
아미노산 1(H) ~ (1,1)
아미노산 2(P) ~ (2,1)
아미노산 3(H) ~ (2,2)
아미노산 4(H) ~ (1,2)
아미노산 5(P) ~ (0,2)
아미노산 6(H) ~ (0,1)

[2차원 격자]
. H P . . .
. H H . . .
. P H . . .
. . . . .
. . . . .
. . . . .
최종 온도: 0.0099998
  
```

그림 II.2.2: 올바른 구조

이렇게 잘 나올 때도 있지만, 제대로 되지 않을 때도 있고,

*복잡도가 2^N 에 비례한다는 것을 기억하자.

```

aqc.cpp -o aqc } ; if ($?) { .\aqc }

=====
❌ 검증 : 물리적으로 유효하지 않음
(사유 : 사슬끊어짐)
=====
도출된 고유구조
아미노산 1(H) ~ (1,0)
아미노산 2(P) ~ (2,0)
아미노산 3(H) ~ (3,0)
아미노산 4(H) ~ (3,1)
아미노산 5(P) ~ (4,2)
아미노산 6(H) ~ (3,2)

[2차원 격자]
. . . . .
H . . . .
P . . . .
H H H . .
. . P . .
. . . . .

최종 온도 : 0.0099998

```

그림 II.2.3: 사슬 끊어짐

물리적으로는 유효하지만 최적의 고유구조가 아닌 경우도 있다.

```

aqc.cpp -o aqc } ; if ($?) { .\aqc }

=====
✅ 검증 : 물리적으로 유효함
=====
도출된 고유구조
아미노산 1(H) ~ (2,2)
아미노산 2(P) ~ (2,1)
아미노산 3(H) ~ (1,1)
아미노산 4(H) ~ (1,2)
아미노산 5(P) ~ (1,3)
아미노산 6(H) ~ (1,4)

[2차원 격자]
. . . . .
. H H P H .
. P H . . .
. . . . .
. . . . .
. . . . .

최종 온도 : 0.0099998

```

그림 II.2.4: 최적의 고유구조가 아님

이렇게 되는 이유는 무엇보다 고전컴퓨터는 양자컴퓨터가 아니기 때문이다. 부차적인 이유로는 최적 수치를 입력하지 못했기 때문도 있을 것이다. 고전컴퓨터는 아미노산 20개로도 고전할 테지만, 양자 컴퓨터는 100개도 거뜰히 계산해낼 수 있다는 점이, 직접 만들어 실행해 보니 새삼스레 신기했다. 양자 컴퓨터가 앞으로 수많은 분야에서 불러올 혁명이 기대된다.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <cmath>
5  #include <random>
6  #include <algorithm>
7
8  using namespace std;
9
10 const int N = 6;           // 아미노산의 개수 (HPHHPH 서열 길이에 맞춤)
11 const int L = 6;           // 격자축의 크기
12 const int M = L * L;       // 총 격자점 수
13 const int K = N * M;       // 총 큐비트 수 (N * M)
14 const double LAMBDA = 5.0; // 고전 SA 페널티 상수는 적당히
15 const double DELTA = 2.0;  // 소수성 결합 보상
16
17 string sequence = "HPHHPH"; // 아미노산 서열
18
19 // 2차원 인덱스를 1차원으로
20 inline int get_idx(int i, int alpha) { return i * M + alpha; }
21
22 // 인접 여부 확인 (맨하튼 거리 = 1)
23 bool is_adjacent(int a, int b)
24 {
25     int r1 = a / L, c1 = a % L;
26     int r2 = b / L, c2 = b % L;
27     return abs(r1 - r2) + abs(c1 - c2) == 1;
28 }
29
30 int main()
31 {
32     vector<vector<double>> W(K, vector<double>(K, 0.0));
33
34     // W matrix
35     // H_1: 하나의 아미노산은 딱 한 곳에만 있어야 한다
36     for (int i = 0; i < N; ++i)
37     {
38         for (int a = 0; a < M; ++a)
39         {
40             int u = get_idx(i, a);
41             W[u][u] += -LAMBDA; // 1차항
42             for (int b = a + 1; b < M; ++b)
43             {
44                 int v = get_idx(i, b);
45                 W[u][v] += 2.0 * LAMBDA; // 2차항
46             }
47         }
48     }
49
50     // H_2: 하나의 격자점에는 두 개 이상 있으면 안된다
51     for (int a = 0; a < M; ++a)
52     {
53         for (int i = 0; i < N; ++i)
54         {
55             for (int j = i + 1; j < N; ++j)
56             {

```

```

57         W[get_idx(i, a)][get_idx(j, a)] += LAMBDA;
58     }
59 }
60 }
61
62 // H_3: 서열 상 이웃하면 격자 상에서 이웃해야한다
63 for (int i = 0; i < N - 1; ++i)
64 {
65     for (int a = 0; a < M; ++a)
66     {
67         for (int b = 0; b < M; ++b)
68         {
69             if (a == b)
70                 continue;
71             if (!is_adjacent(a, b))
72             {
73                 int u = get_idx(i, a), v = get_idx(i + 1, b);
74                 if (u < v)
75                     W[u][v] += LAMBDA;
76                 else
77                     W[v][u] += LAMBDA;
78             }
79         }
80     }
81 }
82
83 // H_4: H-H 붙으면 에너지 낮아지는거
84 for (int i = 0; i < N; ++i)
85 {
86     for (int j = i + 2; j < N; ++j)
87     {
88         if (sequence[i] == 'H' && sequence[j] == 'H')
89         {
90             for (int a = 0; a < M; ++a)
91             {
92                 for (int b = 0; b < M; ++b)
93                 {
94                     if (is_adjacent(a, b))
95                     {
96                         int u = get_idx(i, a), v = get_idx(j, b);
97                         if (u < v)
98                             W[u][v] -= DELTA;
99                         else
100                             W[v][u] -= DELTA;
101                     }
102                 }
103             }
104         }
105     }
106 }
107
108 // 가짜 어닐링
109 vector<int> x(K, 0);
110 random_device rd;
111 mt19937 gen(rd()); // RNG

```

```

112 uniform_real_distribution<> dis(0.0, 1.0);
113 uniform_int_distribution<> bit_dis(0, K - 1);
114
115 double T = 100.0; // 초기 온도
116 double T_min = 0.01; // 최종 온도
117 double alpha = 0.9999; // 냉각 속도
118 int steps = 1500000; // 탐색 횟수
119
120 for (int s = 0; s < steps && T > T_min; ++s)
121 {
122     int k = bit_dis(gen); // 무작위 비트
123
124     double delta_E = 0;
125     int x_k_old = x[k];
126     int x_k_new = 1 - x_k_old;
127     int diff = x_k_new - x_k_old;
128
129     // 대각 성분 1차항 기여
130     delta_E += W[k][k] * diff;
131     // 비대각 성분 2차항 기여
132     for (int j = 0; j < K; ++j)
133     {
134         if (k == j)
135             continue;
136         double weight = (k < j) ? W[k][j] : W[j][k];
137         delta_E += weight * diff * x[j];
138     }
139
140     // Metropolis 수용 조건
141     if (delta_E < 0 || dis(gen) < exp(-delta_E / T))
142     {
143         x[k] = x_k_new;
144     }
145     T *= alpha; // 냉각
146 }
147
148 // 유효성 검증
149 vector<int> pos(N, -1);
150 bool one_hot_ok = true, overlap_ok = true, connectivity_ok = true;
151
152 for (int i = 0; i < N; ++i)
153 {
154     int count = 0;
155     for (int a = 0; a < M; ++a)
156     {
157         if (x[get_idx(i, a)] == 1)
158         {
159             pos[i] = a;
160             count++;
161         }
162     }
163     if (count != 1)
164         one_hot_ok = false;
165 }
166
167 for (int i = 0; i < N; ++i)

```

```

168     {
169         for (int j = i + 1; j < N; ++j)
170         {
171             if (pos[i] != -1 && pos[i] == pos[j])
172                 overlap_ok = false;
173         }
174     }
175
176     for (int i = 0; i < N - 1; ++i)
177     {
178         if (pos[i] == -1 || pos[i + 1] == -1 || !is_adjacent(pos[i], pos[i
179             + 1]))
180             connectivity_ok = false;
181     }
182     // --- 3. 결과 출력 ---
183     cout << "\n===== \n";
184     if (one_hot_ok && overlap_ok && connectivity_ok)
185     {
186         cout << "✅ 검증: 물리적으로 유효함" << "\n";
187     }
188     else
189     {
190         cout << "❌ 검증: 물리적으로 유효하지 않음" << "\n";
191         cout << "    (사유: " << (!one_hot_ok ? "위치오류 " : "")
192             << (!overlap_ok ? "좌표중첩 " : "")
193             << (!connectivity_ok ? "사슬끊어짐 " : "") << ")" << "\n";
194     }
195     cout << "===== \n";
196
197     vector<string> grid(L, string(L, '.'));
198     cout << "도출된 고유구조" << "\n";
199     for (int i = 0; i < N; ++i)
200     {
201         if (pos[i] != -1)
202         {
203             grid[pos[i] / L][pos[i] % L] = sequence[i];
204             cout << "아미노산 " << i + 1 << "(" << sequence[i] << ") ~ (" <<
205                 pos[i] / L << "," << pos[i] % L << ")" << "\n";
206         }
207     }
208     cout << "\n[2차원 격자]\n";
209     for (int r = 0; r < L; ++r)
210     {
211         for (int c = 0; c < L; ++c)
212             cout << grid[r][c] << " ";
213         cout << "\n";
214     }
215
216     cout << "\n최종 온도: " << T << "\n";
217
218     return 0;
219 }

```

03 결론

III.1. 탐구 후 감상

§ III.1.1. 진행 과정에서 느낀 점

HP 격자 모형으로만 단백질 구조를 다룬 것은 단백질의 복잡성을 과도하게 단순화한 것이 많았나 싶기도 하나, 본 탐구의 서론에서 증명했듯 여전히 그 실효성은 있음을 확인했다고 볼 수 있겠습니다. 생명 현상의 치밀한 체계가 정교한 물리 법칙과 진화의 산물이라는 점은 탐구 과정에서 큰 지적 흥미를 자극했습니다.

단순히 양자 컴퓨터의 우수성을 식과 글자로만 접하는 것과, 프로그램으로 구현하는 것을 통해 고전 컴퓨터가 에너지 장벽 앞에서 「찼찼매는」 현상을 목격하며 왜 제대로 된 값이 안 나올까 고민해 보는 것은 실감의 차원이 달랐습니다. 뱀새가 황새 따라하려다가 다리 찢어진다는 속담이 완벽히 들어맞는 상황입니다.... 무차별 대입이나 복잡한 최적화 문제들에서 양자 컴퓨팅이 가질 압도적 우위를 확인하며, 이 기술이 얼마나 대단한 것을 이루어 낼지 새삼스럽게 기대가 되었습니다. 저 또한 그 발전 과정에 조금이나마 일조하고 싶다는 생각도 들었습니다.

§ III.1.2. 사전 및 사후 조사 과정에서 느낀 점

생명과학에 관심이 있고* 궁금한 것들도 많았지만 실제로 수업시간 외에 공부해볼 생각을 하지는 못했었습니다. 물리, 수학, 컴퓨터과학 등의 분야를 선호해 생명과학은 상대적으로 멀게 느껴졌으나, 이번에 탐구를 통해 생물학적 구조를 수학적으로 정립하고 물리적 알고리즘으로 풀어내는 과정에서 interdisciplinary research[†]의 가치를 깨달았습니다.

조사 과정에서 의문이 조금만 꼬리를 물고 이어지면 한국어 자료가 부족하거나 없어졌기에, 생소한 물리, 화학, 생물의 개념들을 이해하기 위해 원서, 논문, 각종 웹페이지 등을 탐독했습니다. 특히 영어와 일본어 위키백과와 전공서적이 많은 도움이 되었습니다. 이러한 과정이 미래의 성장을 견인해 줄 밑거름이 되었다고 생각합니다.

*인간이 자신의 근원과 주위 세계의 본질에 대한 앎을 추구하는 존재라면 관심이 없을 수 있겠습니까 000

[†]다학제 융합? 범교과 융합? 한국말로 뭐라고 해야할지 모르겠습니다

생명과화학 안에도 분자생물학, 유전학, 미생물학, 면역학, 생화학, 생물리학 등등등 많은 분야가 있겠으나 이렇게 조금이라도 건드려 보니 생명과학 전반의 유기적인 연결성을 실감했으며, 생명과학의 다양한 분야들에 새로운 흥미가 돌아났습니다. 모호하게만 느껴지던 아이디어를 하루의 시간과 한 자루의 샤프심을 들여 명확한 수식적 결론으로 도출해낸 이번 경험은 정말 보람차고 가치 있었다고 느낍니다.

§ III.1.3. 단백질 구조 연구와 양자컴퓨팅에 대하여

단백질 구조를 HP 모형 등으로 단순화하여 연구하려는 기조는 1900년대 후반부터 있었고, 이 문제가 처음 양자컴퓨팅과 접목된지도 15년 정도가 되어* 현재는 실제로 공학적으로 구현을 시도하고 상용화를 위해 노력하는 단계입니다. VQE나 AQC를 이용해 단백질의 고유 구조를 알아내는 것을 넘어 최근에는 알파폴드와 양자알고리즘을 결합한 QFold 등의 프로젝트도 등장하여 연구가 가속화되고 있어 대략 10년 후에는 이 도구를 이용해 도출한 유의미한 결과가 있을 것이라고 합니다.

III.2. 향후 연구 계획

§ III.2.1. 당장 드는 생각

HP 모형을 탈피하여 20가지의 아미노산 간 결합을 모두 알아보고 그것을 모두 포괄하는 규칙을 알아내어 AQC 또는 VQE 알고리즘을 만드는 것을 해보고 싶다는 생각이 듭니다. 물론 많은 시간과 노력이 들 것이며 이미 연구진과 기업들이 하고 있는 일이지만, 이번 탐구와 마찬가지로 최대한 도움을 받지 않고(즉 답지를 보지 않고) 제가 직접 해보는 것은 저에게 정말 큰 의미가 있을 것이라고 생각합니다.

또한, 양자컴퓨팅에 대해서는, 최적화 문제를 잘 푼다는 것을 들으면 요즘 세상에 가장 먼저 떠오르는 것은 AI 입니다. LLM의 학습 과정은 의외로 단백질의 구조를 찾아내는 것과 많은 공통점이 있으며 † 양자컴퓨터가 하이브리드로 학습의 부분부분에 관여한다면 현재의 「버블」이라고 불리는 어텐션 기반의 트랜스포머 체계‡에서도 유의미한 진보를 이루어낼 수 있을 것이라고 생각됩니다. 당장에 VQE

*2012년에 하버드 대학교 연구진이 D-Wave 양자 어닐러를 사용해 6개의 아미노산 서열로 된 아주 작은 단백질 모형을 접는 데 성공한 것이 태동이 되었다고 합니다.

†그래서 알파폴드가 단백질 예측에 뛰어난 것일지도 모릅니다

‡토큰 수에 대한 시간복잡도가 $O(N^2)$ 으로, 입력 길이가 길어질수록 계산량이 폭증해 비효율적이고 명칭해집니다.

나 AQC를 사용해 순전파 신경망을 지금보다 더 빠르고 효율적으로 정확히 학습시킬 아이디어가 막 떠오르고 있는 부분입니다. 또한, 양자컴퓨터의 특징에 맞추어 트랜스포머를 능가할 새로운 LLM 알고리즘이 등장한다면 정말로 무서운 강인공지능(AGI)이 생각보다 빠르게 등장할지도 모르는 것이라는 생각이 들었습니다. 이 부분 전반에 대해 깊이 공부, 연구하고 기여하고 싶습니다.

§ III.2.2. 향후 연구에 대한 사후 조사 결과

실제 연구에서는 MJ(Miyazawa-Jernigan) 행렬과 같은 20×20 의 통계적 상호작용 전위 (statistical potential) 행렬을 사용한다고 합니다. 이렇게 하면서도 QUBO 형식을 유지한다고 하는데 흥미로운 부분입니다. 저도 저만의 행렬을 구축해서 죽이 되든 밥이 되든 해보고 싶다는 생각이 듭니다.

특히 양자컴퓨터의 선형대수적 연산* 가속 효과에 주목해 현재 AI의 물리적 한계를 돌파할 수 있을 것이라는 것이 기계학습에 양자컴퓨팅을 도입하자는 QML(quantum machine learning) 학계의 주류 의견이라고 합니다.

III.3. 교과 세특 요약

사실 저는 글을 잘 못 써서 세특 요약도 어떻게 쓸지 잘 모르겠습니다.... 아래는 보고서의 각 장을 요약한 내용입니다.

수업 시간 중 염색체 교차에 따른 유전적 다양성 증가를 학습하며, 발생 가능한 유전자 조합의 수가 고전적 슈퍼컴퓨터의 계산 범위를 넘어선다는 점에 주목함. 이를 계기로 방대한 경우의 수를 가진 생물학적 난제인 『단백질 접힘(Protein Folding) 문제』를 평소 관심을 두던 양자컴퓨팅 기술과 결합하여 해결하고자 하는 융합적 탐구 주제를 설정함.

100개의 아미노산으로 구성된 단백질이 가질 수 있는 이론적 구조의 수가 3^{100} 개에 달하여 무작위 탐색 시 우주의 나이보다 긴 시간이 소요됨을 증명함. 실제 생체 내에서 수 밀리초 내에 접힘이 일어나는 현상(레빈탈의 역설)을 설명하기 위해 에너지가 낮은 경로로 유도되는 접힘 깔때기 모형의 논리를

*거대한 행렬들의 행렬곱 연산 등을 빠르게 처리하는 것

소개함. 단백질 접합 에너지 경로가 울퉁불퉁할 경우 국소적 최적해에 갇히는 쥘쥘뎀 현상과 이를 극복하는 방법을 통계역학의 무작위 에너지 모형을 상정한 에너지 정규분포와 유리전이온도, 헬름홀츠 자유에너지를 통해 증명함.

20종의 아미노산을 소수성(H)과 친수성(P)으로 단순화한 HP 모형을 도입하여 복잡한 생체 분자 시스템을 수학적으로 추상화함. 수용액 내에서 물 분자의 엔트로피 증가를 위해 소수성 아미노산들이 내부로 뭉치는 소수성 붕괴 이론을 깃스 자유에너지 식을 통해 열역학적으로 분석하고 소수성 붕괴가 일어나는 이유를 볼츠만 엔트로피 공식을 이용해 증명함. 연결 상수(μ)와 엔트로피 지수(γ)를 활용하여 가능한 입체 구조의 수(Ω)를 근사 계산하고, 볼츠만 엔트로피 공식을 적용하여 아미노산 개당 발생하는 엔트로피 손실($\approx -1.1NkB$)을 정량적으로 산출함. 2차원 정방 격자 단백질 모형을 사용하여 아미노산 서열을 격자점 위의 배치로 치환하고, 사슬이 자신을 통과할 수 없는 SAW 및 배제 체적 효과를 반영하여 모형의 일반성의 정당성을 확보함.

알고리즘의 효율성을 빅 O 표기법으로 분석하며, 단백질 접합 문제가 다항 시간 내에 해결 불가능한 NP-Hard 범주에 속함을 소개함. 고전 비트 시스템(0 또는 1)의 결정론적 한계를 지적하고, 간단한 큐비트 중첩상태의 상태공간 차원을 나타내는 수식을 통해 큐비트의 중첩과 양자얽힘을 이용한 양자 병렬성의 우위성을 소개함. 복잡한 최적화 문제를 해결하기 위해 단일 정리를 활용한 AQC 방식을 탐구함. 초기 해밀토니안을 충분히 느리게 시간에 따라 변화시켜 최종 상태의 바닥상태를 얻는 과정을 수식 $H(t) = \left(1 - \frac{t}{T}\right)H_0 + \left(\frac{t}{T}\right)H_P$ 로 정립하고, 양자 터널링을 통해 에너지 장벽을 극복하는 과정을 설명함.

단백질의 각 아미노산(i)이 격자점(r)에 위치하는 상태를 이진 변수 $q_{i,r}$ 로 정의하고, 물리적 제약 조건을 해밀토니안 에너지 함수로 변환함. 단백질 사슬의 연속성을 보장하기 위해 한 아미노산은 반드시 하나의 격자점에만 존재해야 한다는 제약과, 서로 다른 두 아미노산이 동일한 격자점을 점유할 수 없다는 배제 체적 제약을 페널티 항으로 설계함. HP 모형의 핵심인 소수성 상호작용을 계산하기 위해, 서열 상 인접하지 않으면서 격자상에서 인접한 두 소수성(H) 아미노산 쌍에 대해 음의 에너지 값을 부여하는 상호작용 항을 구축함. 이를 통해 전체 시스템의 에너지가 최저가 되는 상태가 생물리학적으로 가장 안정적인 고유구조에 대응됨을 해석함.

C++로 양자 어닐링을 모방하는 엔진을 만든 후 탐구 과정에서 이끌어낸 해밀토니안 규칙을 적용하여 아미노산의 HP 서열을 입력하면 최적 구조를 도출해 격자 위에 시각화하여 보여주는 프로그램을 개발함. 프로그램의 출력 결과가 고르지 못하고, 물리적으로 부적절한 결과도 출력되는 것에 대하여 그 이유를 고전컴퓨터의 디지털 비트의 한계와 양자 터널링의 부재 등으로 들고, 그 깨달음을 바탕으로 하이퍼파라미터를 적절히 조정하여 HP로 표현된 6개의 아미노산에 대하여 그 최적 고유구조를 2차원 격자 상에서 나름 준수한 정확도로 출력해내도록 프로그램을 수정하는데 성공함.

단백질 접힘이라는 생물학적 난제를 수학적 모델로 정립하고 물리적 알고리즘으로 해결하는 과정에서 다학제적 융합(interdisciplinary research)의 가치를 성찰함. 국내 자료의 한계를 극복하기 위해 영어와 일본어 원서, 논문, 웹페이지 등을 탐독함. 모호한 가설을 명확한 수식적 결론으로 도출하기 위해 장시간의 수리적 추론을 하였으며, 이를 통해 생명과학 전반의 유기적인 연결성을 실감하고 학문에 대한 시야를 넓히는 계기를 마련함.

단백질 구조 예측 알고리즘과 거대언어모델(LLM) 학습 과정의 유사성을 발견하고, 현재 트랜스포머 체계가 가진 시간복잡도의 한계를 양자 알고리즘(VQE, AQC)으로 돌파할 수 있는 하이브리드 학습 체계를 인식함. 양자 컴퓨팅의 선형대수 연산 가속 효과를 기계학습에 도입하는 양자 기계학습(QML) 분야에 주목하여, 강인공지능(AGI) 구현을 위한 물리적·전산적 토대를 연구하고 관련 분야 기술 발전에 기여하고자 하는 진로 의식을 드러냄.

참고문헌 및 출처

- [1] A. L. Lehinger, *Biochemistry: the molecular basis of cell structure and function*, 2番目版. 1975.
- [2] S. Hansen, 「Die Entdeckung der proteinogenen Aminosäuren von 1805 in Paris bis 1935 in Illinois」, 2015年3月.
- [3] D. L. Nelson, M. M. Cox, と A. L. Lehninger, 「Polypeptides Fold Rapidly by a Stepwise Process」, 2017年.
- [4] 최정모, 「단백질 접힘 이야기」. [Online]. 入手先: <https://horizon.kias.re.kr/21532/>
- [5] T. Hill, *An Introduction to Statistical Thermodynamics*. 1986.
- [6] 土井正男, ソフトマター物理学入門. 2010.
- [7] Polymer Science Learning Center, 「The Glass Transition」. [Online]. 入手先: <https://pslc.ws/macrog/tg.htm>
- [8] J. Bryngelson と P. Wolynes, 「Spin glasses and the statistical mechanics of protein folding」, 1987年.
- [9] 사이언스올 과학백과사전, 「단열정리 (adiabatic theorem)」. [Online]. 入手先: <https://www.scienceall.com/brd/board/390/L/menu/317?brdType=R&bbsSn=170130>
- [10] M. Born と V. Fock, 「Beweis des Adiabatenatzes」, 1928年.
- [11] E. Farhi, J. Goldstone, S. Gutmann, と M. Sipser, 「Quantum Computation by Adiabatic Evolution」, 2000年.
- [12] K. A. Dill, 「Theory for the folding and stability of globular proteins」, 1985年.

- [13] Makoto Ide, Yasushi Maeda, と Hiromi Kitano, 「Effect of Hydrophobicity of Amino Acids on the Structure of Water」, 1997年.
- [14] Basudev Maity, Jiaxin Tian, Tadaomi Furuta, Satoshi Abe, と Takafumi Ueno, 「Structural insights into a single semi-clathrate hydrate formed in a confined environment of porous protein crystal」, 2023年.
- [15] C. Pace, B. Shirley, M. McNutt, と K. Gajiwala, 「Forces contributing to the conformational stability of proteins」, 1996年.